7.6.2 Injecting services

I've mentioned several times in this book that you need to configure various core services to work with ASP.NET Core. Many services are registered automatically, but often, you must add more to use extra features, such as when you called `AddHttpLogging()` in chapter 3 to add request logging to your pipeline.

NOTE Adding services to your application involves registering them with a dependency injection (DI) container. You'll learn all about DI and registering services in chapters 8 and 9.

You can automatically use any registered service in your endpoint handlers, and ASP.NET Core will inject an instance of the service from the DI container. You saw an example in chapter 6 when you used the `LinkGenerator` service in an endpoint handler. `LinkGenerator` is one of the core services registered by `WebApplicationBuilder`, so it's always available, as shown in the following listing.

Listing 7.7 Using the `LinkGenerator` service in an endpoint handler

```
app.MapGet("/links", (LinkGenerator links) =>    ❶
{
    string link = links.GetPathByName("products");
    return $"View the product at {link}";
});
```

❶ The LinkGenerator can be used as a parameter because it's available in the DI container.

Minimal APIs can automatically detect when a service is available in the DI container, but if you want to be explicit, you can also decorate your parameters with the `[FromServices]` attribute:

```
app.MapGet("/links", ([FromServices] LinkGenerator links) =>
```